

Synthext Specification

Guilherme Guimarães, Humberto Enchaki, Ícaro Argonauta

May 14, 2026

Contents

1	Introduction	2
2	The Compiler	2
2.1	Definitions	2
2.2	Voices	2
2.2.1	Volume	2
2.2.2	Octaves	3
2.2.3	Instruments	3
2.2.4	Start Delay	4
2.2.5	BPM	4
2.3	Commands	4
2.3.1	Notes	4
2.3.2	Pauses	4
2.3.3	Volume	4
2.3.4	Instruments	5
2.3.5	Octaves	5
2.3.6	BPM	5
2.3.7	Repeat	5
2.4	Language	5
2.5	The Output File	6
3	Graphical Interface	6
3.1	Text Editor	7
3.2	Audio Playback	7
3.3	Generation Parameters	7
4	Final Requirements	8
4.1	Functional Requirements	8
4.1.1	Compiler	8
4.1.2	Graphical Interface	8
4.2	Non-functional Requirements	8
4.2.1	Compiler	8
4.2.2	Graphical Interface	8
4.2.3	Code Quality	8

1 Introduction

Synthext is a music production application, composed of a compiler, which translates melodic voices in textual representation to MIDI files, and a graphical interface, including a text editor, voice parameter configuration, and playback controls.

2 The Compiler

The Synthext compiler takes as input a text file consisting of multiple voices, or melodic lines. Each line in the file defines a different voice, and a voice is a sequence of commands, such as playing notes and silences, changing instruments etc.. The compiler then translates this input file into a MIDI file containing the corresponding control codes.

2.1 Definitions

- **Song:** The song produced is a mix of all voices in the file, played in synchrony. The song has a global bpm dictating the pace of all the voices.
- **Voice:** A voice (one line in the file) is a sequence of commands written in ascii text. Each voice has independent properties that can be changed throughout the song through commands and can be initially configured outside the file, such as starting instrument and volume. Commands are **not** separated by whitespace. Voices can also specify an initial delay before starting playing.
- **Command:** Usually each command is composed of a single ascii character. Commands can play notes, silences, change the global bpm, and other special actions.
- **Requirements:** The keyword **MUST** indicates a required feature for the implementation to be considered compliant with the specification. The keyword **MAY** indicates an optional but recommended feature for the implementation.

2.2 Voices

Each voice is defined and identified by a line, and the voice numbering starts in 0, so voice 0 is in line 1 (first line), voice 1 in line 2, voice 2, in line 3, etc.

2.2.1 Volume

Each voice will start in a different volume dictated by their **voice number**. The formula for calculating the starting volume of a voice is:

$$100 - 20(\text{voice mod } 4)$$

So, for the first 6 voices:

- voice 0: volume 100
- voice 1: volume 80
- voice 2: volume 60
- voice 3: volume 40

- voice 4: volume 100 (cycles)
- voice 5: volume 80...

Voices volumes can be changed throughout the song using commands. All volumes must always take values between 0 and 127. Applications MAY provide methods to override the starting volumes for specific voices.

2.2.2 Octaves

Each voice will start in a different octave dictated by their **voice number**. The formula for calculating the starting octave of a voice is:

$$6 - \text{voice mod } 4 \tag{1}$$

So, for the first 6 voices:

- voice 0: octave 6
- voice 1: octave 5
- voice 2: octave 4
- voice 3: octave 3
- voice 4: octave 6 (cycles)
- voice 5: octave 5...

If you want a voice to play outside this range, the octave must be adjusted by commands. A voice can be assigned an octave in the range 0 through 9. Applications MAY provide methods to override the starting values for specific voices.

2.2.3 Instruments

Each voice can be assigned an instrument throughout the song. Complying applications MUST provide at least the 128 General MIDI instruments to choose from.

In the case a voice line doesn't contain any commands to set an instrument, the voice will be assigned a default instrument according to the following rules:

- voice mod 4 = 0: Piano (GM 0)
- voice mod 4 = 1: Organ (GM 20)
- voice mod 4 = 2: Harpsichord (GM 6)
- voice mod 4 = 3: Clarinet (GM 71)

The graphical interface MAY provide methods to configure the default instruments for specific voices.

2.2.4 Start Delay

A voice line may have an initial delay before starting. The delay is expressed as a positive integer between square brackets: "[<number>]". The number represents the amount of beats the voice will wait before starting to run its commands.

2.2.5 BPM

All voices share a global BPM. If a voice changes the BPM through commands, all other voices will update their BPM as well. The initial global BPM is 120. Applications MUST provide methods to configure the starting BPM.

2.3 Commands

Each voice is composed of multiple commands that define the actions of the voice throughout the song. The commands are defined in the following tables.

2.3.1 Notes

Characters A-H and Mb play notes:

Text	Action
A	Play note A
B	Play note B
C	Play note C
D	Play note D
E	Play note E
F	Play note F
G	Play Note G
H	Play Note Bb
Mb	Play Note Eb

2.3.2 Pauses

Characters a-h play pauses. Be careful in that if the character 'b' comes before a character 'M', it will be parsed as a note command (Mb).

Text	Action
a-h	Pause for one beat

2.3.3 Volume

The space character doubles the voice volume, saturating if the maximum volume is reached.

Text	Action
<space>	doubles or maxes out volume

2.3.4 Instruments

The following characters assign a specific instrument to the voice. The even digits have a special command: take the sum of the current instrument's MIDI value and the digit and assign the resulting instrument to the voice.

Text	Action
'!	Change to Harmonica (GM #23)
','	Change to Tubular Bells (GM #15)
','	Change to Church Organ (GM #20)
odd digit	Change to Tubular Bells (GM #15)
even digit	Change to GM #(current + digit value)

2.3.5 Octaves

The following characters increase or decrease the octave by one. If the octave exceeds the maximum or minimum octave, return to the default octave for the voice.

Text	Action
'?'	Increase octave
'V'	Decrease octave

2.3.6 BPM

The characters '<' and '>' decrease and increase the global BPM by 10 beats, respectively.

Text	Action
'<'	Decrease BPM by 10
'>'	Increase BPM by 10

2.3.7 Repeat

The following characters's action depend on if the last command was a note or not: if the last command played was a note, repeat that note; else, play a silence beat.

Text	Action
non-note and non-pause consonants	repeat or silence
'O', 'I', 'U'	repeat or silence
remaining characters	repeat or silence

2.4 Language

It's important to emphasize that the language has significant whitespace, so commands don't have whitespace between them, unless it's the "double volume" command.

The Synthext language expressed in BNF:

```
<song>      ::= <delayed_voice> | <delayed_voice> "\n" <song>
<delayed_voice> ::= <delay> <voice> | <voice>
<voice>      ::= <command> | <voice> <command>
<delay>      ::= "[" <natural> "]"
```

```

<command>      ::= <note>      | <pause> | <volume> | <instrument>
                  | <octave> | <bpm>      | <repeat>

<note>          ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "Mb"
<pause>         ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
<volume>        ::= " "
<instrument>    ::= <bells> | <add_midi> | <church_organ> | <harmonica>
<octave>        ::= <dec_octave> | <inc_octave>
<bpm>           ::= <dec_bpm> | <inc_bpm>
<repeat>        ::= <other_consonants> | "I" | "O" | "U"

<bells>         ::= <odd_digit> | ";"
<add_midi>      ::= <even_digit>
<church_organ>  ::= ","
<harmonica>     ::= "!"

<dec_octave>    ::= "?"
<inc_octave>    ::= "V"
<dec_bpm>       ::= "<"
<inc_bpm>       ::= ">"

<natural>       ::= <digit> | <digit> <natural>
<digit>         ::= <odd_digit> | <even_digit>
<odd_digit>     ::= "1" | "3" | "5" | "7" | "9"
<even_digit>    ::= "0" | "2" | "4" | "6" | "8"

```

2.5 The Output File

The generated MIDI file MUST be compatible with the official MIDI file specification. Whether the file is created using multiple or single tracks is implementation defined. The exact mapping of the Synthext language commands to MIDI control codes is also implementation defined.

The generated MIDI file MUST output the "same" song if played by different synthesizers. Here, "same" means the same melodic and harmonic structures, with the same synchrony (the song must have the same "essence"). The specific instrument sounds may be different because of the different sound banks used.

3 Graphical Interface

The graphical interface MUST provide facilities for editing and saving the input file, saving the generated MIDI file, and playback of the generated MIDI file. The interface MAY provide facilities for configuring the initial parameters defined as optional per the language specification.

3.1 Text Editor

The graphical interface **MUST** contain a text editor, with the following minimal requirements:

- loading and saving the input text file
- basic text-editing capabilities, more specifically insertion, removal, selection, copying, cutting and pasting of text.

The text editor **MAY** include additional quality-of-life features, such as:

- syntax highlighting
- line numbering
- showing the beat number of commands in-line
- showing generated MIDI commands in-line
- navigating the file using semantic information

This list is **NOT** definitive, and any other number of extra features may be added, given that they do not interfere with the essential features.

3.2 Audio Playback

The interface **MUST** provide a way to control the playback of the generated audio. More specifically, the following playback controls are mandatory:

- playing the audio
- pausing the audio
- restarting the audio
- seeking the audio to a specific position

Additionally, other features **MAY** be included, such as:

- saving the generated (in formats like .mp3, .ogg, etc)
- controlling the global audio volume
- muting specific voices

This list is **NOT** definitive, and any other number of extra features may be added, given that they do not interfere with the essential features.

3.3 Generation Parameters

The interface **MUST** provide a control for setting the starting BPM of the song. Additionally the interface **MAY** provide controls for overriding specific voice settings (volume, octave, instrument) as defined in the compiler specification.

The interface **MAY** provide methods for the user to use different sound banks. For example, a control to select a specific soundfont file.

4 Final Requirements

Finally, all functional and non-functional requirements are compiled into the following lists:

4.1 Functional Requirements

4.1.1 Compiler

- Translate an input text file into a valid (and portable) generated MIDI file according to the language specification.
- Expose a way to control the starting parameters of the voices.
- Support AT LEAST 8 different voices.

4.1.2 Graphical Interface

- Provide a text editor capable of editing the source file with the basic requirements according to the text editor specification.
- Provide a way to load and save the source file.
- Provide a way to save the generated MIDI file.
- Provide minimal playback controls, according to the audio playback specification.
- Optionally provide methods for overriding specific voice parameters, according to the compiler specification.

4.2 Non-functional Requirements

4.2.1 Compiler

- Ability to gracefully handle malformed input files.
- Must scale well with large files, speed and memory-wise.

4.2.2 Graphical Interface

- Ability to inform the user in a useful way about errors in the input file.
- Text editor must scale well with large files.
- Audio playback must occur without glitching or stuttering.

4.2.3 Code Quality

- Components must be modular and reusable (eg. compiler doesn't need to know about the graphical interface).
- Good code coverage AT LEAST in components not related to the graphical interface.
- Good documentation, including code documentation and project structure documentation (doc comments, component and class diagrams, etc.)